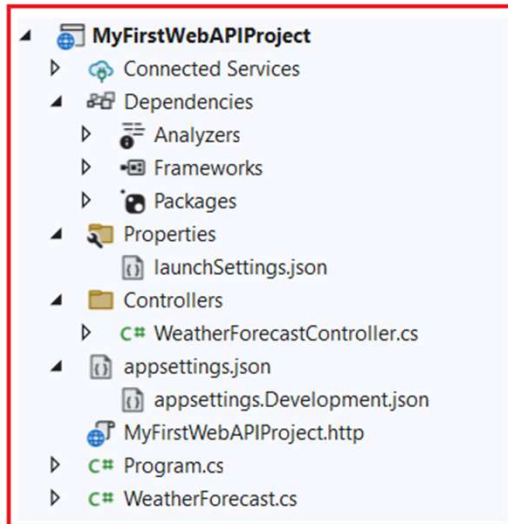


Web API - Projektstruktur



Projektübersicht



- **Program.cs**
Setup (Bootstrapping) des API-Projekts
- **WeatherForecastController.cs**
Exemplarischer Controller - Alle Controller müssen sich im Controllers Ordner befinden
- **WeatherForecast.cs**
Datenmodell für exemplarischen Controller - Typisch im Ordner Models oder ref. Projekt
- **appsettings.json**
wie app.config oder web.config - enthält Application-Settings wie DB-Connection, Umgebungsvariablen, Logging-Konfiguration
- **MyFirstWebAPIProject.http**
Kompatibilität zu anderen IDEs wie VSCode zum Testen und Debuggen von APIs

<https://dotnettutorials.net/lesson/asp-net-core-web-api-files-and-folders/>

HTTP-Profil benutzen (<project-name>.http)

```
@MyFirstWebAPIProject_HostAddress = http://localhost:5222
↓ Click Here to Test this API
✕ Send request | Debug
GET {{MyFirstWebAPIProject_HostAddress}}/weatherforecast/
Accept: application/json
###
```

Vorteile

- Ermöglicht API-Endpunkte direkt aus IDE zu testen ohne externe Tools wie Postman
- Unterliegt Versionskontrolle
- Praktisch als Dokumentation

The screenshot shows an IDE interface with an HTTP client. On the left, the request is defined: a GET request to the endpoint `weatherforecast/` with the `Accept` header set to `application/json`. The response status is `200 OK` with a time of `29.05 ms` and a size of `389 bytes`. The response body is formatted as JSON and shows a list of three weather forecast entries, each with a date, temperature, and summary.

Formatted Raw Headers Request

Body

application/json; charset=utf-8, 389 bytes

```
[
  {
    "date": "2024-02-01",
    "temperature": -10,
    "summary": "Chilly"
  },
  {
    "date": "2024-02-02",
    "temperature": -5,
    "summary": "Scorching"
  },
  {
    "date": "2024-02-03",
    "temperature": 1,
    "summary": "Rainy"
  }
]
```

Setup der App – Program.cs

- Einstiegspunkt der Anwendung
- Seit .Net 8 Standardvorlage für minimales Hostingmodell
- Die App wird in der Regel per Code in der Program-Klasse konfiguriert, erstellt und ausgeführt
- `CreateBuilder()` Methode erstellt eine `WebApplicationBuilder`-Instanz zum Einrichten des Hostings und Konfiguration der App
- `builder.Services` ist ein Container um via DI Dienste zu registrieren

```
public class Program
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // Add services to the container.
        builder.Services.AddControllers();
        builder.Services.AddEndpointsApiExplorer();
        builder.Services.AddSwaggerGen();

        // Konfigurieren Sie das Logging
        builder.Logging.ClearProviders();
        builder.Logging.AddConsole();
        builder.Logging.AddEventLog();

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI();
        }

        app.UseHttpsRedirection();
        app.UseAuthorization();
        app.Run();
    }
}
```

4

Hinzufügen von Diensten zum DI-Container:

Von der App benötigte Dienste wie MVC, Web-API, Entity Framework, benutzerdefinierte Dienste usw. werden beim Dependency-Injection-Container (DI) registriert. Dienste werden dem DI-Container mithilfe der **builder.Services.Add...**-Methoden hinzugefügt. `AddControllers()` wird normalerweise für eine Web-API verwendet, um Unterstützung für Controller und API-bezogene Funktionen hinzuzufügen.

AddEndpointsApiExplorer: Die Methode `AddEndpointsApiExplorer()` wird verwendet, um API Explorer-Dienste zu aktivieren, die zum Generieren der OpenAPI-Spezifikation erforderlich sind. Diese Methode fügt Dienste hinzu, die zum Erkennen von Endpunktmetadaten erforderlich sind. Diese Metadaten sind für Tools wie Swagger unerlässlich, um genaue und nützliche Dokumentationen für Ihre Web-API zu generieren.

AddSwaggerGen: Die Methode `AddSwaggerGen()` fügt dem Projekt Swagger-Generatordienste hinzu. Diese Dienste sind für die Generierung von Swagger-Dokumenten verantwortlich, die die Struktur und Funktionen Ihrer Web-API beschreiben. Mit dieser Methode können Sie die Swagger-Dokumentation anpassen, z. B. den Titel und die Version festlegen und sogar benutzerdefinierte Beschreibungen und Schemata hinzufügen.

Konfigurieren der HTTP-Anforderungspipeline:

Nachdem Sie die App erstellt haben, müssen Sie die HTTP-Anforderungspipeline konfigurieren, indem Sie Middleware-Komponenten hinzufügen. Die **Methode**

builder.Build() erstellt eine Instanz von `WebApplication`, die zum Einrichten der Middleware-Pipeline verwendet wird. Die Methoden **app.Use...** werden verwendet, um Middleware zur Anforderungspipeline der Anwendung hinzuzufügen. Dazu gehören Fehlerbehandlung, Umleitung zu HTTPS, Bereitstellung statischer Dateien, Routing und Autorisierung.

Ausführen der Anwendung:

Schließlich startet die `Run`-Methode die App.

Hinweis: Die herkömmliche Datei `Startup.cs` wird nicht mehr benötigt. Die Datei `Program.cs` enthält jetzt das, was früher sowohl in `Program.cs` als auch in `Startup.cs` enthalten war.

Konfigurationen

- Verwenden von mehreren Umgebungen
 - [launchSettings.json \(MS Docs\)](#)
- [appsettings.json konfigurieren \(MS Docs\)](#)

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "AllowedHosts": "*"   
}
```

```
// Logging Konfigurieren  
builder.Logging.ClearProviders();  
builder.Logging.AddConsole();  
builder.Logging.AddDebug();
```

```
// nur Windows  
builder.Logging.AddEventLog();
```

OpenTelemetry Protocol

- Offener Standard und Framework für das Sammeln, Verarbeiten und Exportieren von Telemetriedaten
- Einheitliche Erfassung von Logs, Metriken und Traces über verschiedene Plattformen und Sprachen hinweg
- Nutzt vorhandene ASP.NET API
- Daten via OTLP an Monitoring-Systeme senden
 - Azure Monitor
 - Prometheus
 - Jaeger
 - [Grafana loki](#)
 - [Seq](#) > [\[dockerhub\]](#)



[Open Telemetry | Getting Started](#)

```
// Install-Package OpenTelemetry.Exporter.Console
builder.Logging.AddOpenTelemetry(options =>
    options.AddConsoleExporter()
);
```

```
// Install-Package OpenTelemetry.Exporter.OpenTelemetryProtocol
var uri = new Uri("http://localhost:5341/ingest/otlp/v1/logs");
builder.Logging.AddOpenTelemetry(options =>
    options.AddOtlpExporter(config => {
        config.Endpoint = uri;
        config.Protocol = OtlpExportProtocol.HttpProtobuf;
        config.Headers = "X-Seq-ApiKey=1234";
    })
);
```

6

```
docker pull datalust/seq
docker run -d --name seq -e ACCEPT_EULA=Y -p 8088:80
-p 5341:5341 [-v /pfad/zu/seq/data:/data]
datalust/seq:latest
docker kill seq
docker rm seq
```

Warum ist das Loki-Setup komplexer als Seq?

Seq ist ein All-in-One-Logging-Server, bei dem du einfach deine Anwendung mit dem Seq-Client konfigurierst und fertig.

Loki hingegen ist modular aufgebaut:

- Promtail sammelt die Logs,
- Loki speichert und verwaltet sie,
- Grafana stellt sie dar.

Das macht Loki flexibler und skalierbarer, aber auch komplexer in der Einrichtung

Zusammengefasst

- Loki = Log-Server (wie Seq, aber Cloud-native und effizienter)
- Promtail = Log-Sammler (Client, schickt Logs an Loki)
- Prometheus = Metriken-Monitoring (keine Logs, nur Zahlenwerte)
- Grafana = Dashboard für alles
- Jaeger (oder alternativ Grafana Tempo) ist für Tracing, also für die Analyse

einzelner Anfragen in verteilten Systemen.

OpenTelemetry Protocol -- Features & Vorteile

- **Logs**
Erfassen und speichern von Ereignissen und Informationen, z.B. Fehlermeldungen und Statusänderungen.
- **Metriken**
Messen von Kennzahlen wie Antwortzeiten, Anzahl der Requests, Serverauslastung.
- **Traces**
Nachverfolgen von Anfragen durch das gesamte System, um Abläufe und Bottlenecks sichtbar zu machen
- **Vendor-Unabhängigkeit**
Keine Bindung an bestimmte Monitoring-Tools, da das OTLP-Protokoll (OpenTelemetry Protocol) verwendet wird.
- **Einfache Integration**
In ASP.NET Web APIs kann OpenTelemetry mit wenigen NuGet-Paketen und Codezeilen integriert werden.
- **Automatische Instrumentierung**
Viele gängige Bibliotheken (z.B. ASP.NET Core, HttpClient, SqlClient) werden automatisch erfasst

Dependency Injection

Inversion of Control (IoC) zwischen
Klassen und Abhängigkeiten

Dependency Injection

- *Dependency Injection* ist ein Spezialgebiet des Inversion of Control Entwurfsmusters.
- Dependency Injection verwendet ein Objekt – *den Container* – zur Initialisierung von Objekten und liefert dem Objekt die geforderten Abhängigkeiten
- Kann in beliebige Projekte installiert werden (NuGet)
 - Standardmäßig installiert



Dependency Injection mittels IoC-Container

```
// Gängigste Variante: Konstruktor Injection
public class ClientController : Controller
{
    private ICarService _carService;

    public Client(ICarService carService)
    {
        if (carService == null)
            throw new ArgumentNullException("carService must not be null");

        _carService = carService;
    }
}
```

- Abhängigkeiten werden hier bspw. über den Konstruktor zur Verfügung gestellt
- Abhängigkeiten werden in einem separaten IoC-Container konfiguriert

Registrierung der Services

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddTransient<ICarService, CarService>();
}
```

AddTransient

Sobald ein *Request* nach einem *Type* erfolgt => neue Instanz

AddSingleton

Eine Instanz für die gesamte *Lifetime* der Anwendung

AddScoped

Jeder *HTTP Request* => neue Instanz

<https://docs.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection>

Controller Class - Action Injection

Mit dem `FromServicesAttribute` ist es möglich, einen Dienst direkt in eine Aktionsmethode einzufügen, ohne eine Constructor Injection verwenden zu müssen

1.)

```
4 Verweise
public interface IDateTime
{
    2 Verweise | 0 Ausnahmen
    DateTime Now { get; }
}
1 Verweis
public class SystemDateTime : IDateTime
{
    2 Verweise | 0 Ausnahmen
    public DateTime Now
    {
        get { return DateTime.Now; }
    }
}
```

2.)

```
// This method gets called by the runtime. Use this method to add services to the container.
0 Verweise | 0 Ausnahmen
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers();
    services.AddSwaggerGen(c =>
    {
        c.SwaggerDoc(name: "v1", info: new OpenApiInfo { Title = "PPEDVDemoWebAPI", Version = "v1" });
    });
    services.AddSingleton<IDateTime, SystemDateTime>();
}
```

3.)

```
[Route(template: "api/[controller]")]
[ApiController]
1 Verweis
public class ClockServiceController : ControllerBase
{
    0 Verweise | 0 Ausnahmen
    public ClockServiceController()
    {
    }

    0 Verweise | 0 Anforderungen | 0 Ausnahmen
    public string GetTime([FromServices] IDateTime dateTime)
    {
        return dateTime.Now.ToLocalTime().ToString();
    }
}
```

DI: Zugreifen auf Einstellungen von einem Controller

- Das in Optionsmuster in ASP.NET Core beschriebene Optionsmuster ist der bevorzugte Ansatz, um Einstellungen zu verwalten.
- Fügen Sie generell IConfiguration nicht direkt in einen Controller ein.

```
1.) public class SampleWebSettings
{
    0 Verweise | 0 Ausnahmen
    public string Title { get; set; }
    0 Verweise | 0 Ausnahmen
    public int Updates { get; set; }
}
```

```
2.) // This method gets called by the runtime. Use this method to add services to the container.
0 Verweise | 0 Ausnahmen
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers();
    services.AddSwaggerGen(c =>
    {
        c.SwaggerDoc("v1", info: new OpenApiInfo { Title = "PPEDVDemoWebAPI", Version = "v1" });
    });
    services.Configure<SampleWebSettings>(Configuration);
}
```

```
3.) public class SettingsController : ControllerBase
{
    private readonly SampleWebSettings _settings;

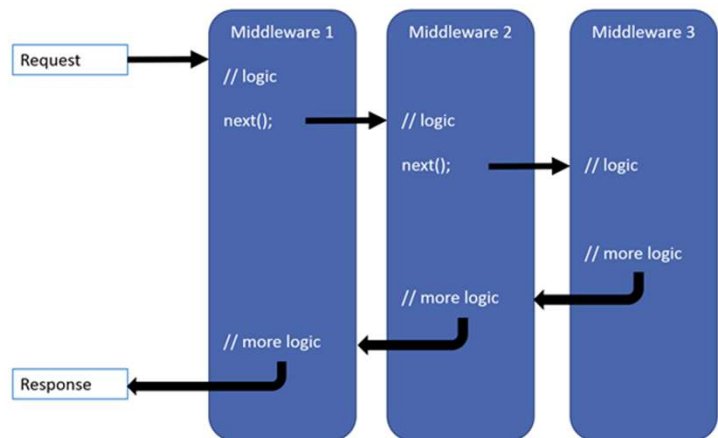
    0 Verweise | 0 Ausnahmen
    public SettingsController(IOptions<SampleWebSettings> settingsOptions)
    {
        _settings = settingsOptions.Value;
    }
}
```

Middleware

Dienste-Schicht, um Komplexität der App zu verbergen

Middleware

- Use-Methoden
 - Bauen einen „Middleware-Pfad“ auf
- Map
 - Fügt Middleware mit einem Pfad hinzu (Routing)
- MapWhen
 - Selbiges wie Map, aber mit Bedingungen
- Run
 - Startet die Application
 - Letztes Statement



<https://docs.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-2.2>

Run(): adds a middleware and ends the pipeline i.e. doesn't call next middleware.

Use(): adds a middleware, as a lambda or a dedicated class.

Map(): adds middleware based on request paths

```
public void Configure(IApplicationBuilder app) {
    app.Run(async context => {
        await context.Response.WriteAsync("Hello, World!"); }); }
....
```

```
app.Use(next =>
{
    return async ctx =>
    {
        await ctx.Response.WriteAsync("before 1");
        await next(ctx);
        await ctx.Response.WriteAsync("after 1");
    };
});
```

```
app.Use(next =>
{
    return async ctx =>
    {
        await ctx.Response.WriteAsync("before 2");
        await next(ctx);
        await ctx.Response.WriteAsync("after 2");
    };
});
```

```
app.Use(next =>
{
    return async ctx =>
    {
        await ctx.Response.WriteAsync("before 3");
        await next(ctx);
        await ctx.Response.WriteAsync("after 3");
    };
});
```

```
app.UseMvc(routes =>
```

Verwendung Environment Variablen

- Environment Name

- IsDevelopment(), IsStaging(), IsProduction(), IsEnvironment(env)

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
        app.UseSwagger();
    }
    else
    {
        app.UseExceptionHandler("/Home/Error");
    }
}
```

Browser Link ?

Fehlerbehandlung

- *Try Catch* in der Controller Action
- Kann in größeren Projekten unübersichtlich werden

```
try
{
    return Ok(_carRepository.GetAll());
}
catch (Exception ex)
{
    return BadRequest();
}
```

Fehlerbehandlung

- *ExceptionHandler Middleware* in Program.cs/Startup.cs
- Guter Ansatz für Production Environment
- Weniger Wartungsaufwand als Try-Catch

```
app.UseExceptionHandler(builder => {  
    builder.Run(async context => {  
        context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;  
  
        var error = context.Features.Get<ExceptionHandlerFeature>();  
        if (error != null)  
        {  
            context.Response.AddApplicationError(error.Error.Message);  
            await context.Response.WriteAsync(error.Error.Message);  
        }  
    });  
});
```

Configuration extern

- Appsettings.json
 - Key Values, nicht case Sensitiv
 - Appsettings.dev
 - ConfigurationBuilder
 - `configuration.GetConnectionString`
- User Secrets
- Enviroment Variablen
- Command Line Argumente

Startup.cs vs. Program.cs

- Vor .NET Core 3.1 Einbindung per Startup.cs
- Heute: Nurnoch Program.cs notwendig
- Startup.cs kann weiterhin verwendet werden

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);  
  
Startup start = new Startup(builder.Configuration);  
start.ConfigureServices(builder.Services);  
  
WebApplication app = builder.Build();  
start.ConfigureMiddleware(app, builder.Environment);
```